11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(1).to be == 1
expect(1).to be < 2
expect(1).to be <= 2
expect(2).to be > 1
expect(2).to be >= 1
expect(String).to be === 'a string'
expect(/foo/).to be =~ 'food'
```

In each case, RSpec uses your operator—such as == or <—to compare the actual and expected values. The matcher on the first line, `be == 1`, is equivalent to `eq(1)`. Use whichever you prefer.

When passing one of these operator matchers into a different matcher, you'll probably want to use the alias `a_value`. For example:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
squares = 1.upto(4).map { |i| i * i }
expect(squares).to include(a_value > 15)
```

So far, we've been comparing precise values like integers and strings. Next, we'll see what happens when we drag imprecise floating-point numbers into the mix.

Delta and Range Comparisons

Floating-point numbers are an unfortunate reality in the world of binary computing with limited precision. Checking two floats for exact equality will frequently cause failures. For example, if you try this seemingly straightforward expectation:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(0.1 + 0.2).to eq(0.3)
```

…then you get a failure:

```
expected: 0.3
     got: 0.30000000000000004

(compared using ==)
```

This failure may surprise you, but you'll see similar behavior in any language that uses IEEE-754 floats, as Ruby does.[2] Just as decimal math can't express most real numbers using a finite amount of digits (for example, 1/3 = 0.333…), your computer's internal binary representation of floating-point numbers is also imperfect.

2. http://0.30000000000000004.com/